Mathematical Models of the Constrained PD-Type Attitude Controller

A learning guide: every equation stated, derived, checked by hand and connected to our drone

Base paper: M. Golestani, K. A. Alattas, S. Ud Din, S. Mobayen, A. K. Alanazi, A. Fekih, "A Simple Structure Constrained Attitude Control for Rigid Bodies: A PD-Type Control", *IEEE Access*, vol. 10, pp. 10202–10209, 2022 (DOI 10.1109/ACCESS.2022.3144627)

Paper model (blocks 1–6)	rigid body → saturation → finite-time envelope → normalised errors → constrained PD law → adaptive estimate
X3 model (blocks 7–10)	pose → body rates → outer position loop → paper law with a sampled-data gain cap → X3 motor mixer

Group AB02 • Introduction to Drones (22AIE448)

Nethra R (CB.SC.U4AIE24147) • Dheeraj S (CB.SC.U4AIE24050)
Raaman Namputhiri (CB.SC.U4AIE24148) • Jyothsna (CB.SC.U4AIE24117)

School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore

Who this is for. Anyone who wants to understand, equation by equation, how the controller of the base paper works and how we put it on a quadcopter. Nothing beyond vectors, matrices and derivatives is assumed: the refresher in section 1.4 covers quaternions, the cross-product matrix and Lyapunov functions. Every toy example can be re-run in MATLAB with `toy_model/run_all.m`, which prints the same numbers. The companion document `docs/complete_documentation` contains the full results, the code walk-through and the claims.

Contents

1 Start Here **2**
1.1 The big picture in plain words . . . 2
1.2 What is special about this controller . . . 2
1.3 How every block is explained . . . 2
1.4 Maths refresher . . . 3

2 Notation, Conventions and Parameters **4**
2.1 Frames and conventions . . . 4
2.2 Master symbol table . . . 5
2.3 The paper's numerical set-up (Section IV) . . . 5
2.4 The X3 quadcopter (Gazebo) parameters . . . 6

3 The Two Equation Flows **6**

4 The Paper's Model: Blocks 1–6 **8**
4.1 Block 1 — How a rigid body turns . . . 8
4.2 Block 2 — What the actuators can really do . . . 9
4.3 Block 3 — The finite-time envelope (FTPPF) . . . 10
4.4 Block 4 — Normalised errors . . . 12
4.5 Block 5 — The constrained PD-type control law . . . 12
4.6 Block 6 — The adaptive estimate . . . 14
4.7 Theorem 1 — Why the closed loop converges . . . 15

5 The Quadcopter Model: Blocks 7–10 **16**
5.1 Block 7 — Body rate from two poses . . . 17
5.2 Block 8 — Outer loop: thrust and the attitude to aim for . . . 18
5.3 Block 9 — The paper's law on a 60 Hz loop . . . 19
5.4 Block 10 — From thrust and torque to four rotor speeds . . . 20

6 Where Every Equation Lives in Our Project **21**
6.1 Which barrier form each file uses . . . 22
6.2 From the equations to a running controller: one control step on the X3 . . . 22

A Recap, Glossary and References **22**
A.1 Recap table . . . 22
A.2 Glossary . . . 23
A.3 References . . . 23

1 Start Here

This guide explains all the mathematics behind our project, from the ground up. The base paper proposes a way to *turn* a rigid body (a satellite, a drone) to a desired orientation while guaranteeing in advance how the motion will look: how far it may overshoot, how fast it must settle and how small the final error must be, and all of this with motors that cannot deliver unlimited torque. We implemented that controller, reproduced the paper's simulations, and then carried it onto a quadcopter (the Gazebo X3).

1.1 The big picture in plain words

Controlling the attitude of a body means answering the same five questions many times per second.

Question	What answers it	Where
How does the body turn at all?	Physics: quaternion kinematics and Euler's equation for a spinning body	block 1
What can the motors really do?	A saturation model: commanded torque is clipped at $u_{\max}$	block 2
How should the motion look?	A time-varying "tube" (envelope) the states must stay inside	block 3
How close am I to the edge of the tube?	Normalised errors: state divided by tube half-width	block 4
Which torque do I apply?	A PD law plus a barrier term that grows near the edge, plus an adaptive term	blocks 5, 6

On the quadcopter four more questions appear: how to get the body rate when Gazebo only publishes the pose (block 7), where the drone should be and therefore how it should be tilted (block 8), how a 60 Hz digital loop changes what the paper's continuous-time law may do (block 9), and how a torque is turned into four rotor speeds (block 10).

1.2 What is special about this controller

Classical *prescribed performance control* (PPC) keeps an error inside a tube by first transforming it into a new variable that goes to infinity at the tube's edge, and then designing a controller for the transformed variable. That works, but the controller then contains partial derivatives of the transformation and becomes long and coupled, and it doubles in complexity when the angular velocity must be constrained as well. The paper's idea is simpler:

1. keep an ordinary PD controller ($-k_p \boldsymbol{q}_v - k_d \boldsymbol{\omega}$) that does the real work of turning the body;
2. *add* one auxiliary term whose gain is small in the middle of the tube and grows without bound near its edge, so the state is pushed back before it can leave;
3. add one scalar adaptive estimate $\hat{\theta}$ that compensates the unknown disturbance and the torque that is lost to saturation;
4. use a new *finite-time* envelope that reaches its final width exactly at a chosen time T_s.

The result is a law with a single line of algebra per axis, which is why the title calls it "simple structure".

1.3 How every block is explained

Each block of the two flow diagrams (section 3) has its own subsection, always in the same order:

Part	What it gives you
Coloured equation box	The equations exactly as in the flow diagram, with the paper's equation numbers
Symbol table	Every symbol: meaning, size (dimension) and unit
Derivation	Where the equation comes from, step by step
Toy example (orange box)	Small numbers you can check with a calculator
In our project (green box)	Where the equation lives in our code and what it does for our drone
Grey box	Which MATLAB step in `toy_model/` prints the same numbers

Three ways to read this guide.

1. *Ten-minute tour:* this section, the two flow diagrams and the recap table (appendix A.1).
2. *Understanding:* only the green "In our project" boxes and the toy examples, in order.
3. *Full learning:* everything in order, running the matching MATLAB step next to each block.

1.4 Maths refresher

Only the tools that are used later appear here, each with the smallest possible example.

Vectors, matrices, norms. A vector $\boldsymbol{a} \in \mathbb{R}^3$ is a column of three numbers. Its length (Euclidean norm) is $\|\boldsymbol{a}\| = \sqrt{a_1^2 + a_2^2 + a_3^2}$. The dot product is $\boldsymbol{a}^\top \boldsymbol{b} = \sum a_i b_i$.

Toy example

$\boldsymbol{s} = [0.13, -0.055, -0.135]^\top$: $\|\boldsymbol{s}\| = \sqrt{0.0169 + 0.003025 + 0.018225} = \sqrt{0.03815} = 0.1953$. This exact vector appears again in blocks 5 and 6.

The cross product as a matrix. For $\boldsymbol{z} = [z_1, z_2, z_3]^\top$ the matrix

$$\boldsymbol{z}^\times = \begin{bmatrix} 0 & -z_3 & z_2 \\ z_3 & 0 & -z_1 \\ -z_2 & z_1 & 0 \end{bmatrix} \tag{1}$$

does the cross product: $\boldsymbol{z}^\times \boldsymbol{y} = \boldsymbol{z} \times \boldsymbol{y}$. It is skew-symmetric, $(\boldsymbol{z}^\times)^\top = -\boldsymbol{z}^\times$, so $\boldsymbol{y}^\top \boldsymbol{z}^\times \boldsymbol{y} = 0$ for every $\boldsymbol{y}$ (a cross product is perpendicular to both factors). This single fact makes the gyroscopic term disappear from the energy balance in Theorem 1.

Toy example

$\boldsymbol{z} = [1, 0, 0]^\top$ gives $\boldsymbol{z}^\times = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$. With $\boldsymbol{y} = [0, 1, 0]^\top$ the three rows give 0, 0 and 1, so $\boldsymbol{z}^\times \boldsymbol{y} = [0, 0, 1]^\top$, which is indeed $\boldsymbol{e}_x \times \boldsymbol{e}_y = \boldsymbol{e}_z$.

Quaternions. A unit quaternion $\boldsymbol{q} = [\boldsymbol{q}_v^\top, q_4]^\top \in \mathbb{R}^4$, $\|\boldsymbol{q}\| = 1$, stores a rotation by an angle ϕ about a unit axis $\boldsymbol{n}$:

$$\boldsymbol{q}_v = \boldsymbol{n} \sin \tfrac{\phi}{2} \in \mathbb{R}^3, \qquad q_4 = \cos \tfrac{\phi}{2} \in \mathbb{R}. \tag{2}$$

So $\boldsymbol{q} = [0, 0, 0, 1]^\top$ is "no rotation", and for small angles $\boldsymbol{q}_v \approx \frac{\phi}{2}\boldsymbol{n}$: *each component of $\boldsymbol{q}_v$ is about half the rotation angle about that axis.* That is why driving $\boldsymbol{q}_v \to 0$ means "reach the target

orientation". Quaternions have no singularities (unlike Euler angles at ±90° pitch), but $\boldsymbol{q}$ and $-\boldsymbol{q}$ are the same rotation, so every implementation keeps the sign continuous between samples. The product of two quaternions (Hamilton product, written $\otimes$) composes two rotations, and the conjugate $\boldsymbol{q}^* = [-\boldsymbol{q}_v^\top, q_4]^\top$ is the inverse rotation.

Toy example

A rotation of 60° about x: $\boldsymbol{q}_v = [\sin 30°, 0, 0] = [0.5, 0, 0]$, $q_4 = \cos 30° = 0.8660$; check $0.5^2 + 0.866^2 = 1.000$. The paper's Part 1 initial attitude $\boldsymbol{q}(0) = [0.2, -0.15, -0.25, 0.9354]^\top$ has $q_4 = 0.9354$, so $\phi = 2\arccos(0.9354) = 41.4°$ away from the target.

Differential equations and RK4. $\dot{x} = dx/dt$ is a rate of change. $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$ tells how fast the state changes, and a computer steps it forward with a small step Δt. Euler's method uses $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \Delta t\, \boldsymbol{f}(\boldsymbol{x}_k)$. The fourth-order Runge–Kutta method (RK4) evaluates the slope four times inside the step and averages them, $\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \frac{\Delta t}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4)$; its error per second of simulation shrinks like Δt^4. All our simulations use RK4 with $\Delta t = 1$ ms.

Saturation and sign functions. $\operatorname{sgn}(x)$ is $+1$, 0 or -1. The saturation function $\operatorname{sat}(x) = \max(-1, \min(1, x))$ equals x in $[-1, 1]$ and is clipped outside. $\operatorname{sat}(x/\varsigma)$ is a smooth replacement for $\operatorname{sgn}(x)$: it reaches ± 1 once $|x| \geq \varsigma$ and is linear in between (a "boundary layer").

Lyapunov functions. To prove that a system settles, find an "energy" $V(\boldsymbol{x}) \geq 0$ that is zero only at the target and show that it can only decrease along the motion, $\dot{V} \leq 0$. A ball in a bowl is the picture: height is V, friction makes it go down, so the ball ends at the bottom.

Toy example

$\dot{x} = -2x$, $V = x^2$: $\dot{V} = 2x\dot{x} = -4x^2 \leq 0$, and $\dot{V} = 0$ only at $x = 0$. Indeed $x(t) = x(0)e^{-2t} \to 0$.

$\mathcal{L}_2$ and Barbalat's lemma. A signal is in $\mathcal{L}_2$ if $\int_0^\infty \|x\|^2 dt < \infty$ (finite "energy"). Barbalat's lemma: if a signal has finite energy *and* its derivative is bounded (so it cannot spike in ever thinner pulses), then the signal itself tends to zero. Theorem 1 of the paper uses exactly this argument.

Positive definite. A symmetric matrix $M \succ 0$ if $\boldsymbol{x}^\top M \boldsymbol{x} > 0$ for every $\boldsymbol{x} \neq 0$. For a symmetric 2×2 matrix this holds when both diagonal entries and the determinant are positive. $\lambda_{\min}(M)$ and $\lambda_{\max}(M)$ are its smallest and largest eigenvalues, and $\lambda_{\min}\|\boldsymbol{x}\|^2 \leq \boldsymbol{x}^\top M \boldsymbol{x} \leq \lambda_{\max}\|\boldsymbol{x}\|^2$.

Eigenvalues and discrete-time stability. For $\boldsymbol{x}_{k+1} = M\boldsymbol{x}_k$ the state decays if every eigenvalue of M has $|\lambda| < 1$ and blows up if one has $|\lambda| > 1$. Block 9 uses this to decide how large a gain a 60 Hz loop can take.

2 Notation, Conventions and Parameters

2.1 Frames and conventions

- **Inertial frame** $\mathcal{F}_i$ (in Gazebo: the world frame, z up) and **body frame** $\mathcal{F}_b$, fixed to the vehicle at its centre of mass. The angular velocity $\boldsymbol{\omega}$ is the rate of $\mathcal{F}_b$ with respect to $\mathcal{F}_i$, *expressed in* $\mathcal{F}_b$ (what a gyroscope measures).

- **Quaternion order.** The paper writes $\boldsymbol{q} = [q_1, q_2, q_3, q_4]^\top$ with the scalar *last*. ROS and Gazebo use $[x, y, z, w]$, also scalar last, so the two agree and no reordering is needed. (In its Lyapunov function the paper calls the scalar part q_0; it is the same q_4.)
- **Element-wise operations.** Inside the barrier term the division and the power act on each axis separately: $(1-\boldsymbol{\varepsilon}_1)^{-2}$ means the vector with entries $(1-\varepsilon_{1i})^{-2}$, and the bracket multiplies $\boldsymbol{s}$ entry by entry (a diagonal gain matrix).

2.2 Master symbol table

Symbol	Meaning	Dim.	Unit	Paper value
$\boldsymbol{q} = [\boldsymbol{q}_v^\top, q_4]^\top$	unit attitude quaternion (error to the target)	$\mathbb{R}^4$	–	$\|\boldsymbol{q}\| = 1$
$\boldsymbol{q}_v$	vector part of $\boldsymbol{q}$	$\mathbb{R}^3$	–	–
$\boldsymbol{\omega}$	body angular velocity	$\mathbb{R}^3$	rad/s	–
J	inertia matrix, symmetric, $J \succ 0$	$\mathbb{R}^{3\times3}$	$\mathrm{kg\,m^2}$	see (3)
$\boldsymbol{u}$	torque commanded by the controller	$\mathbb{R}^3$	N m	–
$\mathrm{sat}(\boldsymbol{u})$	torque the actuators really produce	$\mathbb{R}^3$	N m	$\|\cdot\| \le u_{\max}$
$u_{\max}$	actuator limit per axis	$\mathbb{R}$	N m	2.5
$\boldsymbol{d}$	external disturbance torque, $\|\boldsymbol{d}\| \le \bar{d}$	$\mathbb{R}^3$	N m	see (4)
$\boldsymbol{\Pi} = [\pi_1, \pi_2, \pi_3]^\top$	torque lost to saturation, $\mathrm{sat}(\boldsymbol{u}) - \boldsymbol{u}$	$\mathbb{R}^3$	N m	–
$\boldsymbol{T}_d = \boldsymbol{d} + \boldsymbol{\Pi}$	lumped unknown torque	$\mathbb{R}^3$	N m	–
θ	unknown bound, $\|\boldsymbol{T}_d\| \le \theta$	$\mathbb{R}$	N m	–
$\hat{\theta},\ \tilde{\theta} = \theta - \hat{\theta}$	estimate of θ and its error	$\mathbb{R}$	N m	$\hat{\theta}(0) = 0$
$\rho_q(t),\ \rho_\omega(t)$	envelopes (tube half-widths) for $\boldsymbol{q}_v$ and $\boldsymbol{\omega}$	$\mathbb{R}$	–, rad/s	–
ρ_{q0}, ρ_{qT}	initial and final width of ρ_q	$\mathbb{R}$	–	0.3, 0.008
$\rho_{\omega0}, \rho_{\omega T}$	initial and final width of ρ_ω	$\mathbb{R}$	rad/s	0.4, 0.005
κ	shape parameter of the FTPPF (paper: σ_q, σ_ω)	$\mathbb{R}$	–	2.5
T_s	settling time of the FTPPF (paper: T_f)	$\mathbb{R}$	s	15 and 10
$\boldsymbol{\varepsilon}_1,\ \boldsymbol{\varepsilon}_2$	normalised errors $\boldsymbol{q}_v/\rho_q$, $\boldsymbol{\omega}/\rho_\omega$	$\mathbb{R}^3$	–	$\|\cdot\| < 1$
$\boldsymbol{s} = \boldsymbol{\omega} + c\,\boldsymbol{q}_v$	sliding (composite) variable	$\mathbb{R}^3$	rad/s	–
c	weight of $\boldsymbol{q}_v$ in $\boldsymbol{s}$	$\mathbb{R}$	1/s	*not stated*; we use 0.5
$k_p,\ k_d$	PD gains	$\mathbb{R}$	N m, N m s	40, 20
$\alpha,\ \beta$	barrier weights	$\mathbb{R}$	N m s	1.5, 1.2
$\boldsymbol{u}_{cons}$	auxiliary (constraint) torque	$\mathbb{R}^3$	N m	–
ς	boundary-layer width of $\mathrm{sat}(s/\varsigma)$	$\mathbb{R}$	rad/s	*not stated*; we use 0.02
η_1	adaptation time constant	$\mathbb{R}$	–	0.1
$\eta_2(t)$	leakage, $\int_0^\infty \eta_2\,dt < \infty$	$\mathbb{R}$	1/s	$2e^{-0.5t}$

2.3 The paper's numerical set-up (Section IV)

$$J = \begin{bmatrix} 20 & 1.2 & 0.9 \\ 1.2 & 17 & 1.4 \\ 0.9 & 1.4 & 15 \end{bmatrix} \mathrm{kg\,m^2}, \qquad \lambda(J) = \{14.27,\ 17.00,\ 20.74\}\ \mathrm{kg\,m^2} \tag{3}$$

$$\boldsymbol{d}(t) = \begin{bmatrix} 0.05\sin t, & 0.1\sin 1.2t, & 0.15\sin 1.5t \end{bmatrix}^\top \mathrm{N\,m}, \qquad \|\boldsymbol{d}\| \le 0.187\ \mathrm{N\,m} \tag{4}$$

	Part 1 (effect of T_s)	**Part 2** (constrained vs. conventional PD)
$\boldsymbol{q}(0)$	$[0.2,\ -0.15,\ -0.25,\ 0.9354]^\top$	$[0.3,\ -0.2,\ -0.3,\ 0.8832]^\top$
$\boldsymbol{\omega}(0)$ (rad/s)	$[0.03,\ 0.02,\ -0.01]^\top$	$[-0.02,\ -0.02,\ 0.025]^\top$
T_s	15 s and 10 s	not stated; we use 15 s
ρ_{q0}	0.3	0.3 in the paper; we use 0.36 (see section 2.3)

Four values the paper does not give, and what we chose

1. c in $\boldsymbol{s} = \boldsymbol{\omega} + c\boldsymbol{q}_v$. Theorem 1 needs c "sufficiently small"; the explicit condition (section 4.7) is $c < k_d/\lambda_{\max}(J) = 20/20.74 = 0.964$. We use $c = 0.5$.
2. ς in $\mathrm{sat}(s/\varsigma)$. We use $\varsigma = 0.02\,\mathrm{rad/s}$, small compared with the initial $\|\boldsymbol{s}\| \approx 0.2$.
3. The settling time of Part 2. We use $T_s = 15\,\mathrm{s}$, the first value of Part 1.
4. ρ_{q0} in Part 2. The paper's Part 2 initial attitude has $|q_1(0)| = |q_3(0)| = 0.3 = \rho_{q0}$, so $\varepsilon_{11}(0) = 1$: the state starts *on* the boundary, the barrier $(1-\varepsilon)^{-2}$ is infinite and the paper's own requirement $|q_i(0)| < \rho_{q0}$ (Remark 7) is violated. We use $\rho_{q0} = 1.2\max_i |q_i(0)| = 0.36$.

Every result in this guide and in the complete documentation is reported with these choices, and the sensitivity to them is measured (complete documentation, Section "Sensitivity").

2.4 The X3 quadcopter (Gazebo) parameters

All values are read from the Gazebo model files (`quadcopter.sdf` and the `MulticopterMotorModel` plugin), not tuned. Rotor positions are in the body frame, x forward, y left.

Quantity	**Value**	**Source**
mass m	$1.5\,\mathrm{kg}$ body $+ 4 \times 0.005\,\mathrm{kg}$ rotors $= 1.52\,\mathrm{kg}$	SDF
inertia J about the CoM	$\mathrm{diag}(0.03565,\ 0.07035,\ 0.09892)\,\mathrm{kg\,m^2}$ (body + rotors, parallel-axis)	SDF
rotor 0, 1, 2, 3	$(0.13, -0.22)$ ccw, $(-0.13, 0.20)$ ccw, $(0.13, 0.22)$ cw, $(-0.13, -0.20)$ cw (m)	SDF
thrust constant k_T	$8.54858 \times 10^{-6}\,\mathrm{N/(rad/s)^2}$, $F_i = k_T\Omega_i^2$	plugin
moment constant c_Q	0.016 m, yaw torque $-\mathrm{dir}_i\, c_Q F_i$	plugin
max rotor speed	$800\,\mathrm{rad/s}$, so $F_{\max} = 5.471\,\mathrm{N}$ per rotor	plugin
rotor time constants	12.5 ms (up), 25 ms (down)	plugin
pose rate	≈60 Hz on `/world/quadcopter/dynamic_pose/info`	SceneBroadcaster

Scale. The paper's spacecraft has an inertia about **250 times** larger than the X3 and a torque limit about 7 times larger. The X3 can produce at most 1.46 N m in roll, 0.91 N m in pitch and only **0.106 N m in yaw** while hovering (section 5.4). The paper's gains and envelopes therefore cannot be copied; they must be re-sized (section 5.3).

3 The Two Equation Flows

Everything in this guide is organised along two flow diagrams. Every block shows its equations, what goes **IN** and what comes **OUT**; the arrows carry the signals between blocks. Colours: grey = physics (the plant), orange = the prescribed-performance envelope, magenta = the control law, purple = adaptation, blue = parts that exist only on the quadcopter.

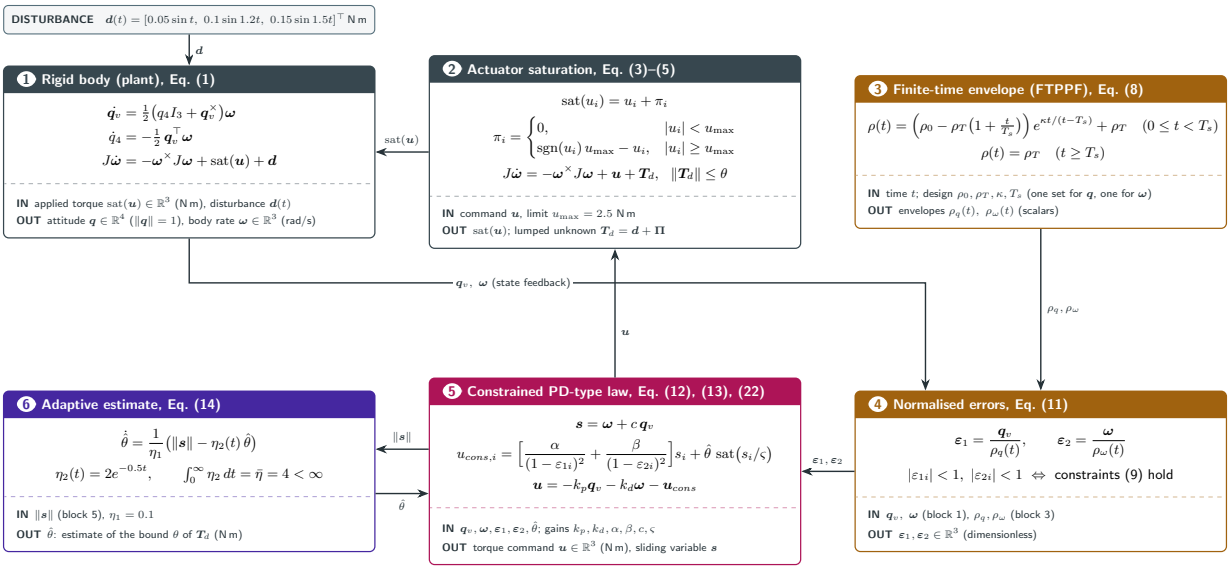


Figure 1: Flow of the base paper (blocks 1–6). The loop is closed through the state feedback $\boldsymbol{q}_v, \boldsymbol{\omega}$. The disturbance $\boldsymbol{d}(t)$ and the saturation (block 2) are the two things the controller does not know.

Reading fig. 1. The plant (block 1) produces the attitude and body rate. The envelope (block 3) says how large they may be at time t; block 4 divides one by the other. The control law (block 5) uses the state, the normalised errors and the adaptive estimate (block 6) to compute a torque. The actuators can only deliver part of it (block 2), and that applied torque, together with the disturbance, turns the body again.

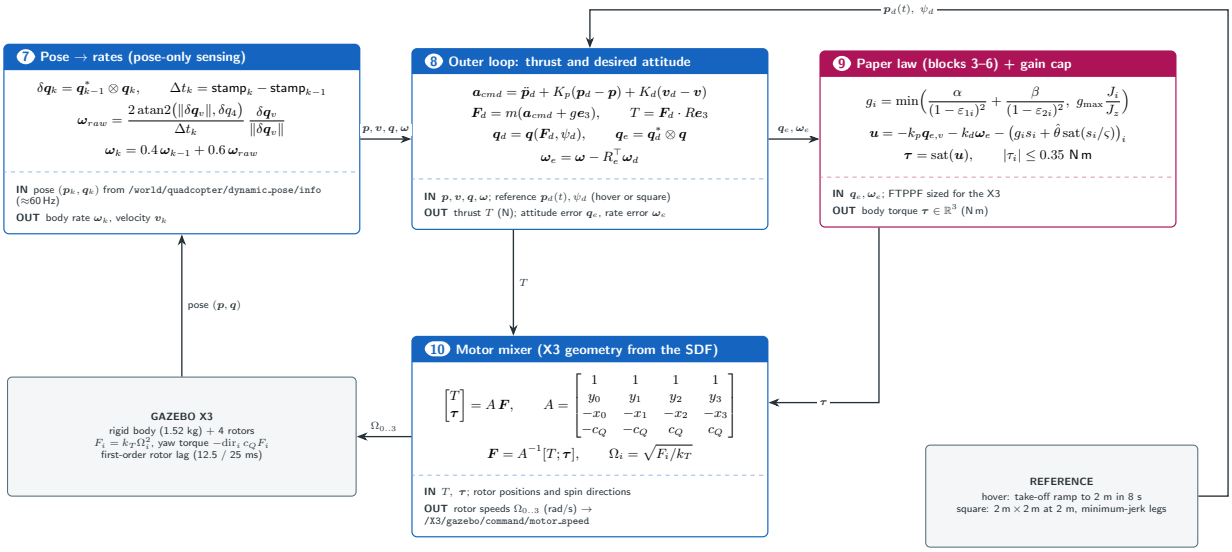


Figure 2: Flow on the Gazebo X3 quadcopter (blocks 7–10). Blocks 3–6 of fig. 1 sit inside block 9 unchanged except for one cap on the barrier gain. Gazebo replaces blocks 1–2: it integrates the real rigid-body and rotor physics.

Reading fig. 2. A quadcopter cannot just turn: it must also hold itself up, and it moves sideways by tilting. So an *outer* loop (block 8) decides the total thrust and the attitude the drone should have, the paper's law (block 9) turns the body to that attitude, and the mixer (block 10) splits thrust and torque into four rotor speeds. Gazebo only publishes the pose, so the body rate is reconstructed from two consecutive quaternions (block 7).

Block	Name	Paper equations	Section
1	Rigid-body attitude dynamics	(1), (2)	4.1
2	Actuator saturation	(3)–(5)	4.2
3	Finite-time prescribed performance function	(6)–(8)	4.3
4	Normalised errors	(9)–(11)	4.4
5	Constrained PD-type control law	(12), (13), (22)	4.5
6	Adaptive law	(14)	4.6
–	Stability proof (Theorem 1)	(15)–(21)	4.7
7	Body rate from the pose	– (ours)	5.1
8	Outer position loop and desired attitude	– (ours)	5.2
9	Paper law on a 60 Hz loop	(12)–(14) + gain cap	5.3
10	X3 motor mixer	– (ours)	5.4

4 The Paper's Model: Blocks 1–6

This section follows fig. 1 from the plant to the adaptive law. All numbers in the toy examples use the paper's Part 1 initial state at $t = 0$, $\boldsymbol{q}(0) = [0.2, -0.15, -0.25, 0.9354]^\top$, $\boldsymbol{\omega}(0) = [0.03, 0.02, -0.01]^\top$ rad/s, so the examples of consecutive blocks chain together: the output of one is the input of the next.

4.1 Block 1 — How a rigid body turns

The idea in plain words

Two separate facts describe a turning body. *Kinematics*: if it spins with rate $\boldsymbol{\omega}$, its orientation changes in a known way (this is pure geometry, no forces). *Dynamics*: a torque changes the spin, and a body that spins about two axes at once starts spinning about the third by itself (gyroscopic coupling).

Equation (1): quaternion kinematics and Euler's rotational dynamics

$$\dot{\boldsymbol{q}}_v = \tfrac{1}{2}\left(q_4 I_3 + \boldsymbol{q}_v^{\times}\right)\boldsymbol{\omega}, \qquad \dot{q}_4 = -\tfrac{1}{2}\,\boldsymbol{q}_v^\top \boldsymbol{\omega} \tag{1a}$$

$$J\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega}^{\times} J\boldsymbol{\omega} + \boldsymbol{u} + \boldsymbol{d} \tag{1b}$$

Symbol	Meaning	Dimension	Unit
$\boldsymbol{q} = [\boldsymbol{q}_v^\top, q_4]^\top$	attitude quaternion	$\mathbb{R}^4$	–
$\boldsymbol{\omega}$	body angular velocity	$\mathbb{R}^3$	rad/s
J	inertia matrix	$\mathbb{R}^{3\times3}$	kg m^2
$\boldsymbol{u}$	control torque (in block 2 replaced by sat($\boldsymbol{u}$))	$\mathbb{R}^3$	N m
$\boldsymbol{d}$	disturbance torque	$\mathbb{R}^3$	N m
I_3	3×3 identity	$\mathbb{R}^{3\times3}$	–

Where (1a) comes from

Rotating first by $\boldsymbol{q}$ and then by a tiny extra rotation $\boldsymbol{\omega}\,dt$ (measured in the body frame) is the quaternion product $\boldsymbol{q}(t + dt) = \boldsymbol{q}(t) \otimes [\frac{1}{2}\boldsymbol{\omega}\,dt;\ 1]$, because by (2) a small angle ϕ about $\boldsymbol{n}$ is $[\frac{\phi}{2}\boldsymbol{n};\ 1]$. Hence $\dot{\boldsymbol{q}} = \frac{1}{2}\,\boldsymbol{q} \otimes [\boldsymbol{\omega}; 0]$. Writing out the Hamilton product $[\boldsymbol{a}; a_4] \otimes [\boldsymbol{b}; b_4] =$

$[a_4\boldsymbol{b} + b_4\boldsymbol{a} + \boldsymbol{a}\times\boldsymbol{b};\ a_4 b_4 - \boldsymbol{a}^\top\boldsymbol{b}]$ with $\boldsymbol{a} = \boldsymbol{q}_v$, $a_4 = q_4$, $\boldsymbol{b} = \boldsymbol{\omega}$, $b_4 = 0$ gives

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\begin{bmatrix} q_4\boldsymbol{\omega} + \boldsymbol{q}_v\times\boldsymbol{\omega} \\ -\boldsymbol{q}_v^\top\boldsymbol{\omega} \end{bmatrix} = \begin{bmatrix} \frac{1}{2}(q_4 I_3 + \boldsymbol{q}_v^\times)\boldsymbol{\omega} \\ -\frac{1}{2}\boldsymbol{q}_v^\top\boldsymbol{\omega} \end{bmatrix},$$

which is (1a). It keeps $\|\boldsymbol{q}\| = 1$: $\frac{d}{dt}\|\boldsymbol{q}\|^2 = 2\boldsymbol{q}_v^\top\dot{\boldsymbol{q}}_v + 2q_4\dot{q}_4 = \boldsymbol{q}_v^\top\boldsymbol{q}_v^\times\boldsymbol{\omega} = 0$, since $\boldsymbol{q}_v^\top(\boldsymbol{q}_v\times\boldsymbol{\omega}) = 0$. (A numerical integrator does not keep it exactly, so every simulation re-normalises $\boldsymbol{q}$ after each step.)

Where (1b) comes from

The angular momentum in the body frame is $\boldsymbol{h} = J\boldsymbol{\omega}$. In the inertial frame Newton's law for rotation is $\frac{d}{dt}\boldsymbol{h}_{\mathcal{F}_i} =$ torque. The body frame itself turns with $\boldsymbol{\omega}$, and the derivative of any vector seen from a turning frame picks up the term $\boldsymbol{\omega}\times(\cdot)$:

$$\tfrac{d}{dt}\boldsymbol{h}\big|_{\mathcal{F}_i} = J\dot{\boldsymbol{\omega}} + \boldsymbol{\omega}\times J\boldsymbol{\omega} = \boldsymbol{u} + \boldsymbol{d} \quad\Longrightarrow\quad J\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega}^\times J\boldsymbol{\omega} + \boldsymbol{u} + \boldsymbol{d}.$$

The term $-\boldsymbol{\omega}^\times J\boldsymbol{\omega}$ is the gyroscopic coupling. It does no work: $\boldsymbol{\omega}^\top(\boldsymbol{\omega}^\times J\boldsymbol{\omega}) = 0$ by the skew-symmetry of (1). This is used in the proof of Theorem 1.

Toy example — one evaluation of Eq. (1) at the Part-1 initial state

$q_4\boldsymbol{\omega} = 0.9354\,[0.03, 0.02, -0.01] = [0.02806,\ 0.01871,\ -0.00935]$ and $\boldsymbol{q}_v\times\boldsymbol{\omega} = [0.0065,\ -0.0055,\ 0.0085]$. Their sum times $\frac{1}{2}$:

$$\dot{\boldsymbol{q}}_v = [0.01728,\ 0.00660,\ -0.00043]^\top\ \mathrm{s}^{-1}, \qquad \dot{q}_4 = -\tfrac{1}{2}\boldsymbol{q}_v^\top\boldsymbol{\omega} = -0.00275\ \mathrm{s}^{-1}.$$

Dynamics with the torque the controller will apply at $t = 0$ (block 5 shows it saturates at $[-2.5, 2.5, 2.5]\,\mathrm{N\,m}$) and $\boldsymbol{d}(0) = 0$: $J\boldsymbol{\omega} = [0.615,\ 0.362,\ -0.095]$, $\boldsymbol{\omega}\times J\boldsymbol{\omega} = [0.00172,\ -0.0033,\ -0.00144]$, and

$$\dot{\boldsymbol{\omega}} = J^{-1}\big(-\boldsymbol{\omega}\times J\boldsymbol{\omega} + \mathrm{sat}(\boldsymbol{u})\big) = [-0.141,\ 0.144,\ 0.162]^\top\ \mathrm{rad/s^2}.$$

Even at full torque the heavy spacecraft accelerates by only $\approx 0.15\,\mathrm{rad/s^2}$. Remember this number: it decides which envelopes are feasible (complete documentation, Monte-Carlo study C).

In our project

`paper_reproduction.py`, function `rhs()`, evaluates (1a)–(1b) for a whole batch of simulations at once and is integrated with RK4 at $\Delta t = 1\,\mathrm{ms}$; MATLAB `step6_spacecraft_closed_loop.m` contains an independent re-implementation. On the X3 we do *not* integrate (1): Gazebo does, for the real quadcopter (section 5).

4.2 Block 2 — What the actuators can really do

The idea in plain words

Reaction wheels or rotors have a maximum torque. Whatever the controller asks for beyond it is simply not delivered. The paper does not ignore this: it treats the missing torque as one more unknown disturbance.

Equations (3)–(5): saturation and the lumped unknown torque

$$\mathrm{sat}(u_i) = u_i + \pi_i, \qquad \pi_i = \begin{cases} 0, & |u_i| < u_{\max} \\ \mathrm{sgn}(u_i)\, u_{\max} - u_i, & |u_i| \geq u_{\max} \end{cases} \tag{3–4}$$

$$J\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega}^{\times} J\boldsymbol{\omega} + \boldsymbol{u} + \boldsymbol{T}_d, \qquad \boldsymbol{T}_d = \boldsymbol{d} + \boldsymbol{\Pi}, \quad \|\boldsymbol{T}_d\| \leq \theta \tag{5}$$

Symbol	Meaning	Dimension	Unit
$u_{\max}$	torque limit of each axis	$\mathbb{R}$	N m
π_i	torque lost on axis i	$\mathbb{R}$	N m
$\boldsymbol{T}_d$	disturbance plus lost torque	$\mathbb{R}^3$	N m
θ	an (unknown) upper bound of $\|\boldsymbol{T}_d\|$	$\mathbb{R}$	N m

Derivation

(3)–(4) is just $\mathrm{sat}(u_i) = \mathrm{sgn}(u_i)\min(|u_i|, u_{\max})$ written as "command plus error". Putting $\mathrm{sat}(\boldsymbol{u}) = \boldsymbol{u} + \boldsymbol{\Pi}$ into (1b) and collecting everything the controller does not know into $\boldsymbol{T}_d$ gives (5). The trick is that now the plant looks as if it received the *commanded* torque, plus one bounded unknown. The adaptive law (block 6) will estimate a bound for it. θ is bounded because $\boldsymbol{d}$ is bounded and because $\boldsymbol{u}$ stays bounded (all its terms are bounded while the state is inside the envelope).

Toy example — the torque lost at $t = 0$

Block 5 will compute $\boldsymbol{u}(0) = [-10.537,\ 5.710,\ 10.414]^{\top}$ N m. With $u_{\max} = 2.5$ every axis saturates: $\mathrm{sat}(\boldsymbol{u}) = [-2.5,\ 2.5,\ 2.5]$ and $\boldsymbol{\Pi} = \mathrm{sat}(\boldsymbol{u}) - \boldsymbol{u} = [8.037,\ -3.210,\ -7.914]$ N m. At the very start the "unknown" torque is dominated by saturation, not by the disturbance (whose size is at most 0.187 N m).

In our project

`sat_vec()` in `paper_reproduction.py` clips each axis at 2.5 N m; on the X3 the torque is clipped at 0.35 N m and, in addition, the mixer must keep every rotor between 0 and 800 rad/s (block 10).

4.3 Block 3 — The finite-time envelope (FTPPF)

The idea in plain words

Draw a tube around zero that starts wide and narrows with time. If the state never touches the walls of the tube, then automatically: its overshoot is smaller than the initial width, it is at least as fast as the walls close, and its final error is smaller than the final width. The paper's new tube reaches its final width *exactly* at a chosen time T_s and then stays constant, instead of only approaching it.

Equations (6)–(8): performance constraint and finite-time prescribed performance function

$$-\rho_q(t) < q_i(t) < \rho_q(t), \qquad -\rho_\omega(t) < \omega_i(t) < \rho_\omega(t), \qquad i = 1, 2, 3 \tag{6, 9}$$

$$\rho(t) = \begin{cases} \left(\rho_0 - \rho_T\left(1 + \frac{t}{T_s}\right)\right) \exp\left(\frac{\kappa\, t}{t - T_s}\right) + \rho_T, & 0 \leq t < T_s \\ \rho_T, & t \geq T_s \end{cases} \tag{8}$$

For comparison, the classical exponential PPF is $\rho(t) = (\rho_0 - \rho_\infty)e^{-\kappa t} + \rho_\infty$ (Eq. 7).

Symbol	Meaning	Dimension	Unit
ρ_0	initial width: the largest allowed $\lvert q_i\rvert$ (or $\lvert\omega_i\rvert$)	$\mathbb{R}$	–, rad/s
ρ_T	final width: the guaranteed accuracy after T_s	$\mathbb{R}$	–, rad/s
T_s	settling time of the envelope	$\mathbb{R}$	s
κ	shape: how early the envelope starts to close	$\mathbb{R}$	–

Checking the three properties

1. *Start.* At $t = 0$ the exponent is 0, so $\rho(0) = (\rho_0 - \rho_T) \cdot 1 + \rho_T = \rho_0$.
2. *Arrival exactly at T_s.* For $t \to T_s^-$ the denominator $t - T_s \to 0^-$ while the numerator $\kappa t > 0$, so the exponent $\to -\infty$ and $\exp(\cdot) \to 0$. The bracket stays finite ($\to \rho_0 - 2\rho_T$), hence $\rho \to \rho_T$, and the second line continues it. So ρ is continuous and equals ρ_T from T_s on. The exponential PPF instead only gets there as $t \to \infty$.
3. *It never increases and never dips below ρ_T.* If $\rho_0 > 2\rho_T$ (true for every envelope we use), the bracket $b(t) = \rho_0 - \rho_T(1 + t/T_s)$ is positive on $[0, T_s)$, so $\rho > \rho_T$ there. Differentiating,

$$\dot\rho(t) = \Big(-\frac{\rho_T}{T_s} - b(t)\,\frac{\kappa T_s}{(t - T_s)^2} \Big) \exp\Big(\frac{\kappa t}{t - T_s}\Big) < 0,$$

because $\frac{d}{dt}\frac{\kappa t}{t-T_s} = -\frac{\kappa T_s}{(t-T_s)^2}$. Both terms are negative.

Toy example — $\rho_q(2)$ with the paper's attitude envelope, $T_s = 15$ s

$\rho_0 = 0.3$, $\rho_T = 0.008$, $\kappa = 2.5$:

$$b(2) = 0.3 - 0.008\Big(1 + \tfrac{2}{15}\Big) = 0.290933, \qquad \exp\Big(\tfrac{2.5\cdot 2}{2-15}\Big) = e^{-0.38462} = 0.680712,$$

$$\rho_q(2) = 0.290933 \times 0.680712 + 0.008 = 0.206042.$$

The same for the rate envelope ($\rho_0 = 0.4$, $\rho_T = 0.005$): $\rho_\omega(2) = 0.394333 \times 0.680712 + 0.005 = 0.273428$ rad/s. Values along the way (from the same formula): $\rho_q = 0.300,\ 0.252,\ 0.206,\ 0.091,\ 0.0245,\ 0.0099,\ 0.0080$ at $t = 0, 1, 2, 5, 8, 10, 12$ s. With $T_s = 10$ s instead, $\rho_q(5) = 0.0316$: a shorter T_s closes the tube faster.

Toy example — finite time versus exponential, $T_s = 8$ s

Exponential PPF with $\kappa = 0.5$ at $t = 8$ s: $(0.3 - 0.008)e^{-4} + 0.008 = 0.01335$, still $1.67 \times \rho_T$. The FTPPF is exactly 0.008 at 8 s.

MATLAB: `toy_model/step1_ftppf.m`. Part A prints the three numbers 0.290933, 0.680712, 0.206042; Part B checks $\rho(0) = \rho_0$, $\rho(T_s) = \rho_T$, monotonicity, and plots the effect of T_s and κ. Library function: `toy_model/lib/ftppf.m`.

In our project

`ftppf()` in `paper_reproduction.py` (vectorised) and in `x3_sim/controllers.py` (scalar, with a guard that returns ρ_T once the exponent is below -50 to avoid underflow). The paper's values are used for the spacecraft; on the X3 the envelope is re-sized to $\rho_q : 0.30 \to 0.02$, $\rho_\omega : 2.5 \to 0.05$ rad/s, $\kappa = 2$, $T_s = 10$ s (section 5.3).

4.4 Block 4 — Normalised errors

The idea in plain words

Instead of transforming the error through a complicated function (classical PPC), simply divide it by the current half-width of the tube. The result says "what fraction of the allowed room is used". Inside the tube means $|\varepsilon| < 1$.

Equation (11): the simple error transformation

$$\boldsymbol{\varepsilon}_1 = \frac{\boldsymbol{q}_v(t)}{\rho_q(t)}, \qquad \boldsymbol{\varepsilon}_2 = \frac{\boldsymbol{\omega}(t)}{\rho_\omega(t)}, \qquad (9) \text{ holds} \iff |\varepsilon_{1i}(t)| < 1 \text{ and } |\varepsilon_{2i}(t)| < 1 \ \ \forall i \tag{11}$$

Symbol	Meaning	Dimension	Unit
$\boldsymbol{\varepsilon}_1 = [\varepsilon_{11}, \varepsilon_{12}, \varepsilon_{13}]^\top$	attitude error as a fraction of ρ_q	$\mathbb{R}^3$	–
$\boldsymbol{\varepsilon}_2 = [\varepsilon_{21}, \varepsilon_{22}, \varepsilon_{23}]^\top$	rate error as a fraction of ρ_ω	$\mathbb{R}^3$	–

Why this is enough

$\rho_q, \rho_\omega > 0$ always, so dividing by them does not change signs and $|q_i| < \rho_q \iff |q_i/\rho_q| < 1$. Nothing has to be differentiated: classical PPC would need $\partial S^{-1}/\partial(\cdot)$ and $\dot{\rho}$ inside the controller. A necessary condition is that the state *starts* inside: $|q_i(0)| < \rho_{q0}$ and $|\omega_i(0)| < \rho_{\omega 0}$ (paper Remark 7).

Toy example — Part-1 state at $t = 0$ ($\rho_q = 0.3$, $\rho_\omega = 0.4$)

$$\boldsymbol{\varepsilon}_1 = \frac{[0.2,\ -0.15,\ -0.25]}{0.3} = [0.6667,\ -0.5,\ -0.8333],$$
$$\boldsymbol{\varepsilon}_2 = \frac{[0.03,\ 0.02,\ -0.01]}{0.4} = [0.075,\ 0.05,\ -0.025].$$

All six are below 1 in magnitude: the start is admissible. Axis 3 uses 83 % of its attitude room.

Part-2 state with the paper's $\rho_{q0} = 0.3$: $\varepsilon_{11} = 0.3/0.3 = 1$ exactly. The constraint is violated at $t = 0$ and $(1 - \varepsilon_{11})^{-2} = 1/0$. This is why we use $\rho_{q0} = 0.36$ for Part 2 (section 2.3); then $\boldsymbol{\varepsilon}_1(0) = [0.833, -0.556, -0.833]$.

MATLAB: `toy_model/step2_normalised_error.m`, Part A, prints $\boldsymbol{\varepsilon}_1$, $\boldsymbol{\varepsilon}_2$ and the gains of block 5.

4.5 Block 5 — The constrained PD-type control law

The idea in plain words

A spring and a damper: the PD part pulls the attitude error to zero ($-k_p \boldsymbol{q}_v$, the spring) and brakes the rotation ($-k_d \boldsymbol{\omega}$, the damper). On top of that, a second, *variable* damper acts on the combination $\boldsymbol{s} = \boldsymbol{\omega} + c\boldsymbol{q}_v$: its strength is small in the middle of the tube and becomes huge near the wall. Finally $\hat{\theta}\ \mathrm{sat}(\boldsymbol{s}/\varsigma)$ adds a push of size $\hat{\theta}$ against the unknown torque.

Equations (12), (13) and (22): the controller

$$\boldsymbol{u} = -k_p \boldsymbol{q}_v - k_d \boldsymbol{\omega} - \boldsymbol{u}_{cons} \tag{12}$$

$$\boldsymbol{u}_{cons} = \Big[\alpha(1-\boldsymbol{\varepsilon}_1)^{-2} + \beta(1-\boldsymbol{\varepsilon}_2)^{-2}\Big]\boldsymbol{s} + \hat{\theta}\ \mathrm{sgn}(\boldsymbol{s}), \qquad \boldsymbol{s} = \boldsymbol{\omega} + c\,\boldsymbol{q}_v \tag{13}$$

$$u_{cons,i} = \left[\frac{\alpha}{(1-\varepsilon_{1i})^2} + \frac{\beta}{(1-\varepsilon_{2i})^2}\right] s_i + \hat{\theta}\ \mathrm{sat}\Big(\frac{s_i}{\varsigma}\Big) \quad \text{(Remark 5, chattering-free form)} \tag{22}$$

Symbol	Meaning	Dimension	Unit
$\boldsymbol{s}$	sliding variable, a "target" rate $-c\boldsymbol{q}_v$ compared with $\boldsymbol{\omega}$	$\mathbb{R}^3$	rad/s
$g_i = \frac{\alpha}{(1-\varepsilon_{1i})^2} + \frac{\beta}{(1-\varepsilon_{2i})^2}$	barrier gain of axis i	$\mathbb{R}$	N m s
$k_p,\ k_d$	PD gains (40, 20)	$\mathbb{R}$	N m, N m s
$\alpha,\ \beta$	barrier weights (1.5, 1.2)	$\mathbb{R}$	N m s
c	weight of $\boldsymbol{q}_v$ in $\boldsymbol{s}$ (0.5)	$\mathbb{R}$	s^{-1}
ς	boundary-layer width (0.02)	$\mathbb{R}$	rad/s

How the barrier term keeps the state inside

1. In the middle of the tube ($\boldsymbol{\varepsilon} \approx 0$) the gain is $g_i \approx \alpha + \beta = 2.7$, small next to $k_d = 20$: the controller behaves like plain PD.
2. As the state approaches the wall, $\varepsilon \to 1$ and $(1-\varepsilon)^{-2} \to \infty$. The term $-g_i s_i$ then dominates. Because $\boldsymbol{s} = \boldsymbol{\omega} + c\boldsymbol{q}_v$, a large negative feedback on s_i drives ω_i towards $-cq_i$, i.e. it makes the body rotate *back towards zero* and caps its speed. This is Remark 2/Remark 7 of the paper.
3. $\boldsymbol{s}$ is the natural variable: when $\boldsymbol{s} = 0$ we have $\boldsymbol{\omega} = -c\boldsymbol{q}_v$, which by (1a) makes $\boldsymbol{q}_v$ decay ($\frac{d}{dt}\|\boldsymbol{q}_v\|^2 = 2\boldsymbol{q}_v^\top \dot{\boldsymbol{q}}_v = q_4 \boldsymbol{q}_v^\top \boldsymbol{\omega} = -c\,q_4 \|\boldsymbol{q}_v\|^2 < 0$ for $q_4 > 0$). Pushing $\boldsymbol{s}$ to zero therefore also pushes the attitude error to zero.

A detail the paper leaves open: one-sided or two-sided barrier?

Written literally, $(1-\varepsilon_{1i})^{-2}$ grows only as $\varepsilon_{1i} \to +1$. As $\varepsilon_{1i} \to -1$ it tends to $1/4$. So the paper's barrier protects the upper wall of the tube much more strongly than the lower wall. A symmetric alternative is $\alpha(1-\varepsilon_{1i}^2)^{-2}$, which grows at both walls; our first browser and Gazebo code used it. We treat the **paper form as the reference** and report the symmetric form as a clearly labelled variant ("symmetric barrier") everywhere. Both are compared in the complete documentation.

Toy example — the torque at $t = 0$ for the Part-1 state, $\hat{\theta}(0) = 0$

Sliding variable: $\boldsymbol{s} = \boldsymbol{\omega} + 0.5\,\boldsymbol{q}_v = [0.03 + 0.1,\ 0.02 - 0.075,\ -0.01 - 0.125] = [0.13,\ -0.055,\ -0.135]$ rad/s.

PD part: $-40\boldsymbol{q}_v - 20\boldsymbol{\omega} = [-8 + (-0.6),\ 6 - 0.4,\ 10 + 0.2] = [-8.6,\ 5.6,\ 10.2]$ N m.

Barrier gains (paper form) with $\boldsymbol{\varepsilon}_1, \boldsymbol{\varepsilon}_2$ from block 4:

$$g_1 = \frac{1.5}{(1-0.6667)^2} + \frac{1.2}{(1-0.075)^2} = 13.500 + 1.403 = 14.903, \quad g_2 = 1.996, \quad g_3 = 1.588.$$

$\boldsymbol{u}_{cons} = g \odot \boldsymbol{s} = [1.937,\ -0.110,\ -0.214]$, and

$$\boldsymbol{u} = [-8.6,\ 5.6,\ 10.2] - [1.937,\ -0.110,\ -0.214] = [-10.537,\ 5.710,\ 10.414]\ \mathrm{N\,m}.$$

Every axis exceeds 2.5 N m, so $\mathrm{sat}(\boldsymbol{u}) = [-2.5,\ 2.5,\ 2.5]$ (block 2). Note the one-sided effect:

axis 3 is *closest* to its wall ($|\varepsilon_{13}| = 0.833$) but gets the *smallest* gain, because $\varepsilon_{13} < 0$. With the symmetric form the gains would be $[6.07,\ 3.87,\ 17.27]$ and $\boldsymbol{u} = [-9.39,\ 5.81,\ 12.53]$.

MATLAB: `step2_normalised_error.m` prints the gains g of both forms and plots them over ε; `step3_control_law.m` prints $\boldsymbol{s}$, $\boldsymbol{u}_{PD}$, $\boldsymbol{u}_{cons}$, $\boldsymbol{u}$ and $\mathrm{sat}(\boldsymbol{u})$ for the three controllers. Library function: `toy_model/lib/control_law.m`.

In our project

`control()` in `paper_reproduction.py` implements (12) and (22) with `mode = "constrained"` (paper form), `"symmetric"` or `"pd"` ($\boldsymbol{u}_{cons} = 0$). The value of ε fed to the barrier is limited to 0.999; this guard is never active while the state is inside the envelope and only prevents a division by zero if it ever leaves.

paper_reproduction.py — control(): Eq. (11), (12), (22)

```
117 def control(t, q, w, th, p, mode):
118     """Eq. (12)-(13) with Eq. (22) boundary layer. Returns u (unsaturated), s, eps1, eps2."""
119     qv = q[:, :3]
120     rq = ftppf(np.array([t]), p["rho_q0"], p["rho_qT"], p["kappa_q"], p["Ts"])[0]
121     rw = ftppf(np.array([t]), p["rho_w0"], p["rho_wT"], p["kappa_w"], p["Ts"])[0]
122     e1 = qv / rq
123     e2 = w / rw
124     s = w + p["c"] * qv
125     u = -p["kp"] * qv - p["kd"] * w
126     if mode == "constrained":            # paper form, Eq. (13)
127         e1c = np.minimum(e1, 0.999)       # numerical guard only (never active when |eps|<1)
128         e2c = np.minimum(e2, 0.999)
129         gain = p["alpha"] / (1 - e1c) ** 2 + p["beta"] / (1 - e2c) ** 2
130         u = u - (gain * s + th[:, None] * np.clip(s / p["varsigma"], -1, 1))
131     elif mode == "symmetric":            # variant used in the original browser/Gazebo code
132         e1c = np.clip(e1, -0.999, 0.999)
133         e2c = np.clip(e2, -0.999, 0.999)
134         gain = p["alpha"] / (1 - e1c ** 2) ** 2 + p["beta"] / (1 - e2c ** 2) ** 2
135         u = u - (gain * s + th[:, None] * np.clip(s / p["varsigma"], -1, 1))
136     elif mode == "pd":
137         pass
138     else:
139         raise ValueError(mode)
140     return u, s, e1, e2, rq, rw
```

4.6 Block 6 — The adaptive estimate

The idea in plain words

The controller does not know how large the disturbance plus the lost torque is. So it learns a number $\hat{\theta}$: it grows while the sliding variable is not zero ("something is still pushing me away") and is slowly pulled back by a leakage term. $\hat{\theta}$ then sets the size of the push $\hat{\theta}\ \mathrm{sat}(\boldsymbol{s}/\varsigma)$ in block 5.

Equation (14): adaptive law

$$\dot{\hat{\theta}} = \frac{1}{\eta_1}\Big(\|\boldsymbol{s}\| - \eta_2(t)\,\hat{\theta}\Big), \qquad \eta_1 > 0, \qquad \int_0^\infty \eta_2(\tau)\,d\tau = \bar{\eta} < \infty \tag{14}$$

Paper choice: $\eta_1 = 0.1$, $\eta_2(t) = 2e^{-0.5t}$ (so $\bar{\eta} = 2/0.5 = 4$), $\hat{\theta}(0) = 0$.

Reading the law

Without leakage ($\eta_2 = 0$), $\hat{\theta}$ is the time integral of $\|\boldsymbol{s}\|/\eta_1$: it keeps growing as long as $\boldsymbol{s} \neq 0$. The leakage $-\eta_2\hat{\theta}$ pulls it down. The paper requires η_2 to have a *finite* integral, which means the leakage dies out with time; this is what makes the extra term in the Lyapunov inequality bounded (section 4.7).

Toy example — one Euler step, $\Delta t = 1$ ms, from $\hat{\theta} = 0$

$\|\boldsymbol{s}(0)\| = \sqrt{0.13^2 + 0.055^2 + 0.135^2} = 0.19532$. Then $\hat{\theta}(1\,\text{ms}) = 0 + 0.001 \times (0.19532 - 2\cdot 0)/0.1 = 0.0019532\,\text{N m}$.

What the leakage that dies out means in practice

With the chattering-free form (22) the sliding variable does not become exactly zero: inside the boundary layer $\hat{\theta}\ \text{sat}(s_i/\varsigma)$ is smaller than $\hat{\theta}$, so a small residual $\|\boldsymbol{s}\|$ remains while the disturbance acts. Because $\eta_2 \to 0$, nothing then stops $\hat{\theta}$ from creeping upwards slowly ("parameter drift"). We observe exactly this in the reproduction: $\hat{\theta}$ is still rising at $t = 20\,\text{s}$ (≈ 0.43), although the state has long converged. It is harmless over the paper's horizon, but on the X3, which runs for 180 s, we keep $\eta_2 \geq 0.2$ and $\hat{\theta} \leq 0.1\,\text{N m}$. `step4_adaptive_law.m` shows the three cases side by side.

In our project

In `paper_reproduction.py` the law is integrated with explicit Euler at 1 ms after each RK4 step, with the projection $\hat{\theta} \geq 0$ (an estimate of a bound cannot be negative):

paper_reproduction.py — adaptive law, Eq. (14)

```
190         if mode != "pd":
191             # adaptive law, Eq. (14) (explicit Euler; theta_hat >= 0 projection)
192             th = np.maximum(th + dt * (np.linalg.norm(s, axis=1) - eta2_fun(t) * th) / p["
    eta1"], 0.0)
```

4.7 Theorem 1 — Why the closed loop converges

Theorem 1 of the paper and its Lyapunov function (15)

For the system (5) under the control (12)–(14), $\boldsymbol{q}_v(t) \to 0$ and $\boldsymbol{\omega}(t) \to 0$ as $t \to \infty$. Proof candidate:

$$V = (k_p + ck_d)\big(\boldsymbol{q}_v^\top \boldsymbol{q}_v + (1 - q_4)^2\big) + \tfrac{1}{2}\boldsymbol{\omega}^\top J\boldsymbol{\omega} + c\,\boldsymbol{q}_v^\top J\boldsymbol{\omega} + \tfrac{\eta_1}{2}\,\tilde{\theta}^2, \qquad \tilde{\theta} = \theta - \hat{\theta} \tag{15}$$

The argument in four steps

1. V **is a valid "energy".** Using $|c\,\boldsymbol{q}_v^\top J\boldsymbol{\omega}| \leq c\,\lambda_{\max}(J)\|\boldsymbol{q}_v\|\|\boldsymbol{\omega}\|$ and $\tfrac{1}{2}\boldsymbol{\omega}^\top J\boldsymbol{\omega} \geq \tfrac{1}{2}\lambda_{\min}(J)\|\boldsymbol{\omega}\|^2$, $V \geq \tfrac{1}{2}\boldsymbol{x}^\top P\boldsymbol{x}$ with $\boldsymbol{x} = [\|\boldsymbol{q}_v\|,\ \|\boldsymbol{\omega}\|,\ |\tilde{\theta}|]^\top$ and

$$P = \begin{bmatrix} 2(k_p + ck_d) & -c\lambda_{\max}(J) & 0 \\ -c\lambda_{\max}(J) & \lambda_{\min}(J) & 0 \\ 0 & 0 & \eta_1 \end{bmatrix}.$$

$P \succ 0$ needs $2(k_p + ck_d)\lambda_{\min}(J) > (c\lambda_{\max}(J))^2$.

2. **Differentiate along the motion.** Using (1a), (5), (12), the skew-symmetry of $\boldsymbol{\omega}^\times$ and $\dot{\tilde{\theta}} = -\dot{\hat{\theta}}$, the paper obtains (Eq. 17)

$$\dot{V} = -ck_p\|\boldsymbol{q}_v\|^2 - k_d\|\boldsymbol{\omega}\|^2 + \boldsymbol{s}^\top\boldsymbol{T}_d + \tfrac{c}{2}\big(q_4\boldsymbol{\omega} + \boldsymbol{\omega}^\times\boldsymbol{q}_v\big)^\top J\boldsymbol{\omega} - \boldsymbol{s}^\top\big[\cdots\big]\boldsymbol{s} - \|\boldsymbol{s}\|\hat{\theta} - \tilde{\theta}\big(\|\boldsymbol{s}\| - \eta_2\hat{\theta}\big).$$

   The barrier term $-\boldsymbol{s}^\top[\cdots]\boldsymbol{s}$ is ≤ 0 because its gain is positive (Remark 2) and can be dropped. Since $\boldsymbol{s}^\top\boldsymbol{T}_d \le \|\boldsymbol{s}\|\theta$, the terms with $\|\boldsymbol{s}\|$ combine to $\|\boldsymbol{s}\|(\theta - \hat{\theta} - \tilde{\theta}) = 0$.

3. **Bound the leakage term** with $\tilde{\theta}\hat{\theta} \le -\frac{2\sigma-1}{2\sigma}\tilde{\theta}^2 + \frac{\sigma}{2}\theta^2$ (Eq. 18), which gives $\dot{V} \le -\boldsymbol{x}^\top Q\boldsymbol{x} + \frac{\eta_2\sigma}{2}\theta^2$ with $Q = \operatorname{diag}\big(ck_p,\ k_d - c\lambda_{\max}(J),\ \eta_2\frac{2\sigma-1}{2\sigma}\big)$ (Eq. 19). $Q \succ 0$ needs $c < k_d/\lambda_{\max}(J)$ and $\sigma > \frac{1}{2}$.

4. **Integrate and use Barbalat.** $V(t) \le V(0) - \lambda_{\min}(Q)\int_0^t \|\boldsymbol{x}\|^2 + \frac{\sigma}{2}\theta^2\bar{\eta}$, so V stays bounded, $\boldsymbol{q}_v, \boldsymbol{\omega} \in \mathcal{L}_2$, their derivatives are bounded, and Barbalat's lemma gives $\boldsymbol{q}_v \to 0$, $\boldsymbol{\omega} \to 0$.

Toy example — the two conditions on c with the paper's numbers

$\lambda_{\min}(J) = 14.27$, $\lambda_{\max}(J) = 20.74$, $k_p = 40$, $k_d = 20$, $c = 0.5$:

$$Q: \quad k_d - c\lambda_{\max} = 20 - 10.37 = 9.63 > 0 \qquad (\text{i.e. } c < 20/20.74 = 0.964),$$

$$P: \quad 2(40+10)\cdot 14.27 = 1427 > (0.5\cdot 20.74)^2 = 107.5.$$

Both hold, so our unstated choice $c = 0.5$ satisfies the theorem.

Three things a careful reader notices in the proof

1. **What is proved.** Theorem 1 proves *convergence to the origin*. It does not separately prove that $|\varepsilon_{li}(t)| < 1$ is preserved for all t (that the state never leaves the tube); the paper supports that with Remarks 2 and 7 and with simulations. We therefore phrase our own claims as "constraint satisfaction observed in simulation" and measure it.
2. **Printing details of Eq. (15).** The paper prints $(1 - q_0^2)$ and $\frac{1}{\eta_1}\tilde{\theta}^2$. With $(1 - q_0)^2$ (the form of the cited reference [42]) and the weight $\frac{\eta_1}{2}$, differentiating gives exactly the terms of Eq. (17); we write (15) in that consistent form. The conclusions do not change.
3. **The proof uses** $\operatorname{sgn}(\boldsymbol{s})$**.** With the boundary layer (22) the cancellation in step 2 is exact only for $|s_i| \ge \varsigma$; inside the layer a residual of order ς remains. The practical guarantee is convergence to a small neighbourhood of zero, and $\hat{\theta}$ can drift (block 6). This matches what all our simulations show: the final $\|\boldsymbol{\omega}\|$ is about 0.004 rad/s, not exactly zero.

In our project

`lyapunov()` in `paper_reproduction.py` evaluates the state part of (15) along the Part 2 trajectory (figure `fig08_theta_hat_and_lyapunov.png`). The $\tilde{\theta}$ term cannot be plotted, because the true bound θ of $\boldsymbol{T}_d$ is unknown.

5 The Quadcopter Model: Blocks 7–10

The paper controls a body that can produce torque about every axis directly and does not need to hold itself up. A quadcopter is different in three ways: (i) its four rotors produce one thrust along the body z-axis plus three torques, so attitude and position are coupled (it moves sideways by tilting); (ii) Gazebo publishes only the *pose*, not the angular velocity; (iii) the controller runs as a sampled loop at the pose rate (≈60 Hz) with a transport delay. Blocks 7–10 are what we added so that blocks 3–6 can be used unchanged on the X3. First, the physics that Gazebo integrates (and that our software-in-the-loop surrogate `x3_sim/x3_model.py` reproduces).

The X3 quadcopter as Gazebo simulates it

$$m\ddot{\boldsymbol{p}} = R\begin{bmatrix}0\\0\\\sum_i F_i\end{bmatrix} - mg\,\boldsymbol{e}_3 - \Big(\sum_i |\Omega_i|\Big)c_D\,\boldsymbol{v}_\perp, \qquad F_i = k_T\Omega_i^2 \tag{X1}$$

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\,\boldsymbol{q}\otimes[\boldsymbol{\omega};0], \qquad J\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega}^\times J\boldsymbol{\omega} + \boldsymbol{\tau}, \qquad \boldsymbol{\tau} = \sum_i \Big(\boldsymbol{r}_i \times F_i\boldsymbol{e}_3 - \mathrm{dir}_i\, c_Q F_i\,\boldsymbol{e}_3\Big) \tag{X2}$$

$$\dot{\Omega}_i = \frac{\Omega_{i,cmd} - \Omega_i}{\tau_{\uparrow/\downarrow}}, \qquad \tau_\uparrow = 12.5\ \text{ms},\ \tau_\downarrow = 25\ \text{ms}, \qquad 0 \le \Omega_{i,cmd} \le 800\ \text{rad/s} \tag{X3}$$

Symbol	Meaning	Dimension	Unit
$\boldsymbol{p},\ \boldsymbol{v}$	position and velocity of the CoM (world)	$\mathbb{R}^3$	m, m/s
R	rotation matrix of $\boldsymbol{q}$ (body → world)	$\mathbb{R}^{3\times3}$	–
$F_i,\ \Omega_i$	thrust and speed of rotor i	$\mathbb{R}$	N, rad/s
$\boldsymbol{r}_i$	position of rotor i in the body frame	$\mathbb{R}^3$	m
dir_i	$+1$ counter-clockwise, -1 clockwise rotor	$\mathbb{R}$	–
c_D	rotor drag coefficient (8.06×10^{-5})	$\mathbb{R}$	$\mathrm{N\,s^2/(m\,rad)}$
$\boldsymbol{v}_\perp$	velocity component perpendicular to the rotor axis	$\mathbb{R}^3$	m/s

(X2) is exactly the paper's Eq. (1) with $\boldsymbol{u}+\boldsymbol{d}$ replaced by the torque of the rotors: the attitude part of the X3 *is* the paper's plant, with a 250 times smaller inertia. (X1) is what the paper does not have.

5.1 Block 7 — Body rate from two poses

The idea in plain words

If we know the orientation now and one sample ago, the small rotation between them, divided by the time between them, is the body rate. The small rotation must be computed *in the body frame* ($\boldsymbol{q}_{k-1}^*\otimes\boldsymbol{q}_k$), not by subtracting quaternion components.

Block 7: exact body rate from consecutive quaternions

$$\delta\boldsymbol{q}_k = \boldsymbol{q}_{k-1}^*\otimes\boldsymbol{q}_k, \qquad \boldsymbol{\omega}_{raw} = \frac{2\,\mathrm{atan2}\big(\|\delta\boldsymbol{q}_v\|,\,\delta q_4\big)}{\Delta t_k}\,\frac{\delta\boldsymbol{q}_v}{\|\delta\boldsymbol{q}_v\|}, \qquad \boldsymbol{\omega}_k = 0.4\,\boldsymbol{\omega}_{k-1} + 0.6\,\boldsymbol{\omega}_{raw} \tag{B7}$$

Δt_k is the difference of the *message time stamps* (simulation time), never of the wall clock.

Derivation

From (1a) over one sample with constant $\boldsymbol{\omega}$, $\boldsymbol{q}_k = \boldsymbol{q}_{k-1}\otimes\delta\boldsymbol{q}$ with $\delta\boldsymbol{q} = [\boldsymbol{n}\sin\frac{\phi}{2};\cos\frac{\phi}{2}]$ and $\phi = \|\boldsymbol{\omega}\|\Delta t$, $\boldsymbol{n} = \boldsymbol{\omega}/\|\boldsymbol{\omega}\|$. Multiplying on the left by $\boldsymbol{q}_{k-1}^*$ isolates $\delta\boldsymbol{q}$; its angle is $\phi = 2\,\mathrm{atan2}(\|\delta\boldsymbol{q}_v\|,\delta q_4)$ and its axis is $\delta\boldsymbol{q}_v/\|\delta\boldsymbol{q}_v\|$, so $\boldsymbol{\omega} = \phi\,\boldsymbol{n}/\Delta t$. The first-order filter with weight 0.6 removes sampling noise with a lag of about $\frac{0.4}{0.6}\Delta t \approx 11\,\text{ms}$ at 60 Hz.

Toy example — why the naive formula fails

The drone is yawed by 90°, $\boldsymbol{q}_{k-1} = [0,0,0.7071,0.7071]$, and rolls about its own x-axis at 0.6 rad/s. After $\Delta t = 1/60\,\text{s}$ it has turned by 0.01 rad: $\boldsymbol{q}_k = [0.00354,\ 0.00354,\ 0.70710,\ 0.70710]$.

- Exact (B7): $\delta\boldsymbol{q} = [0.0050, 0, 0, 1.0000]$, angle 0.0100 rad about x, so $\boldsymbol{\omega} = [0.600,\ 0,\ 0]$ rad/s.
- Naive $2(\boldsymbol{q}_{k,v} - \boldsymbol{q}_{k-1,v})/\Delta t$ (our first Gazebo node): $[0.424,\ 0.424,\ -0.001]$ rad/s: the roll is reported as half roll, half pitch. The naive formula equals the body rate only when the vehicle has no yaw.

In our project

`FixedController.update()` in `x3_sim/controllers.py`, section "F11". The first node, `legacy/gazebo_v1.py`, used the naive difference with a filter weight of 0.10, i.e. a lag of about $9\Delta t \approx 150$ ms.

5.2 Block 8 — Outer loop: thrust and the attitude to aim for

The idea in plain words

To hover at a point the drone needs (a) a total thrust that holds it up and (b) a tilt that points part of the thrust towards the point. A PD law on position gives the acceleration we want; that acceleration plus gravity gives the direction the thrust must point in, which fixes the desired attitude. The paper's law then only has to turn the body to this attitude: its "error quaternion" is the rotation from the desired to the actual attitude.

Block 8: position loop, thrust and desired attitude

$$\boldsymbol{a}_{cmd} = \ddot{\boldsymbol{p}}_d + K_p(\boldsymbol{p}_d - \boldsymbol{p}) + K_d(\boldsymbol{v}_d - \boldsymbol{v}) \quad \text{(horizontal part limited to a tilt of 0.35 rad)} \tag{B8a}$$

$$\boldsymbol{F}_d = m(\boldsymbol{a}_{cmd} + g\boldsymbol{e}_3), \qquad T = \boldsymbol{F}_d^{\top} R\,\boldsymbol{e}_3 \tag{B8b}$$

$$\boldsymbol{z}_{b,d} = \frac{\boldsymbol{F}_d}{\|\boldsymbol{F}_d\|}, \quad \boldsymbol{y}_{b,d} = \frac{\boldsymbol{z}_{b,d} \times \boldsymbol{x}_c}{\|\boldsymbol{z}_{b,d} \times \boldsymbol{x}_c\|}, \quad \boldsymbol{x}_{b,d} = \boldsymbol{y}_{b,d} \times \boldsymbol{z}_{b,d}, \quad \boldsymbol{x}_c = [\cos\psi_d, \sin\psi_d, 0]^{\top} \tag{B8c}$$

$$\boldsymbol{q}_e = \boldsymbol{q}_d^{*} \otimes \boldsymbol{q}, \qquad \boldsymbol{\omega}_e = \boldsymbol{\omega} - R_e^{\top}\boldsymbol{\omega}_d \tag{B8d}$$

Gains: $K_{p,z} = 5$, $K_{d,z} = 3.5$ (vertical, acceleration clipped to $\pm 3\,\mathrm{m/s^2}$), $K_{p,xy} = 1.2$, $K_{d,xy} = 1.6$.

Derivation

Projecting (X1) on the world axes and neglecting the small rotor drag, $m\ddot{\boldsymbol{p}} = T\,R\boldsymbol{e}_3 - mg\boldsymbol{e}_3$. To get the acceleration $\boldsymbol{a}_{cmd}$ we need $T\,R\boldsymbol{e}_3 = m(\boldsymbol{a}_{cmd} + g\boldsymbol{e}_3) = \boldsymbol{F}_d$: the body z-axis must point along $\boldsymbol{F}_d$ (B8c), and the thrust is the part of $\boldsymbol{F}_d$ along the *current* body axis (B8b), so a tilted drone does not over-thrust. The desired yaw ψ_d fixes the remaining rotation about $\boldsymbol{z}_{b,d}$. With $\boldsymbol{q}_e$ as the attitude error, $\boldsymbol{q}_v \to \boldsymbol{q}_{e,v}$ in every equation of blocks 3–6, and $\boldsymbol{\omega}_e$ adds the rate of the reference as a feed-forward (it is zero in hover).

Toy example — a 10 cm position error in hover

$\boldsymbol{p}_d - \boldsymbol{p} = [0.1, 0, 0]$ m, $\boldsymbol{v} = 0$: $\boldsymbol{a}_{cmd} = [0.12,\ 0,\ 0]\,\mathrm{m/s^2}$, so $\boldsymbol{F}_d = 1.52\,[0.12,\ 0,\ 9.81] = [0.182,\ 0,\ 14.911]$ N. The thrust axis must lean by $\arctan(0.182/14.911) = 0.70°$ towards $+x$, i.e. $\boldsymbol{q}_d = [0,\ 0.00612,\ 0,\ 0.99998]$ (a pitch of 0.70°), and $T = 14.91$ N, which is the hover thrust $mg = 1.52 \times 9.81 = 14.911$ N.

5.3 Block 9 — The paper's law on a 60 Hz loop

The idea in plain words

The paper's law is written in continuous time: its barrier gain may grow without bound as $|\varepsilon| \to 1$. A digital loop that only sees the drone every 1/60 s and acts one sample late cannot use an unbounded derivative gain: past a certain value it overshoots more every sample and oscillates. So the sum of all gains acting on $\boldsymbol{s}$ must stay below the largest value the sampled loop tolerates. We also re-size the envelope and the gains for a body 250 times lighter.

Block 9: blocks 3–6 on the X3

$$g_i = \min\Big(\frac{\alpha}{(1-\varepsilon_{1i})^2} + \frac{\beta}{(1-\varepsilon_{2i})^2},\ \ g_{\max}\frac{J_i}{J_z}\Big), \qquad \boldsymbol{u} = -k_p\boldsymbol{q}_{e,v} - k_d\boldsymbol{\omega}_e - \big[g_i s_i + \hat{\theta}\ \mathrm{sat}(s_i/\varsigma)\big]_i \tag{B9a}$$

$$\dot{\hat{\theta}} = \frac{1}{\eta_1}\big(\|\boldsymbol{s}\| - \max(2e^{-0.5t}, 0.2)\,\hat{\theta}\big), \qquad 0 \le \hat{\theta} \le 0.1, \qquad \boldsymbol{\tau} = \mathrm{sat}_{0.35}(\boldsymbol{u}) \tag{B9b}$$

X3 values: $k_p = 4$, $k_d = 0.4$, $\alpha = 0.15$, $\beta = 0.12$, $c = 1$, $\varsigma = 0.5$, $\eta_1 = 5$, $g_{\max} = 1$; envelopes $\rho_q : 0.30 \to 0.02$, $\rho_\omega : 2.5 \to 0.05\,\mathrm{rad/s}$, $\kappa = 2$, $T_s = 10\,\mathrm{s}$.

Sizing the gains

For small angles $\boldsymbol{q}_{e,v} \approx \frac{1}{2}\boldsymbol{\phi}$, so one axis of the PD part is $J\ddot{\phi} = -\frac{k_p}{2}\phi - k_d\dot{\phi}$, a mass–spring–damper with $\omega_n = \sqrt{k_p/(2J)}$ and $\zeta = k_d/(2J\omega_n)$. For the paper ($J \approx 17$, $k_p = 40$, $k_d = 20$): $\omega_n = 1.08\,\mathrm{rad/s}$, $\zeta = 0.54$. For the X3 roll axis ($J = 0.0357$, $k_p = 4$, $k_d = 0.4$): $\omega_n = 7.5\,\mathrm{rad/s}$, $\zeta = 0.75$ — about seven times faster, as a small drone must be, and better damped.

The sampled-data limit

One axis, zero-order hold $T = 1/60\,\mathrm{s}$, one sample of delay: with $\boldsymbol{x}_k = [\phi_k; \dot{\phi}_k]$,

$$\boldsymbol{x}_{k+1} = \underbrace{\begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix}}_{A_d}\boldsymbol{x}_k + \underbrace{\begin{bmatrix} T^2/(2J) \\ T/J \end{bmatrix}}_{B_d} u_{k-1}, \qquad u_k = -\tfrac{k_p}{2}\phi_k - k_{d,tot}\dot{\phi}_k.$$

Stacking $[\boldsymbol{x}_k; u_{k-1}]$ gives a 3×3 closed-loop matrix $\begin{bmatrix} A_d & B_d \\ -\frac{k_p}{2} & -k_{d,tot} \quad 0 \end{bmatrix}$; the loop is stable when all its eigenvalues satisfy $|\lambda| < 1$. $k_{d,tot} = k_d + g_i$ is the total gain on the rate, because the barrier multiplies $\boldsymbol{s} = \boldsymbol{\omega} + c\boldsymbol{q}_v$.

Toy example — how large may the barrier gain become?

Roll axis, $J = 0.03565$: $B_d = [T^2/(2J);\ T/J] = [0.003896;\ 0.4675]$. Scanning $k_{d,tot}$, the spectral radius stays below 1 for $0.06 \le k_{d,tot} \le \mathbf{2.12}\,\mathrm{N\,m\,s}$; for pitch the limit is 4.20 and for yaw 5.91. With the cap, the largest total gains are $0.4 + 1.0 \cdot 0.360 = 0.76$ (roll), $0.4 + 0.711 = 1.11$ (pitch) and 1.40 (yaw): all well inside. Without the cap, $\alpha/(1-\varepsilon)^2$ alone reaches $0.15/(1-0.99)^2 = 1500$ at $\varepsilon = 0.99$, far beyond any limit.

MATLAB: `toy_model/step8_sampled_data_limit.m` builds the matrix above and plots the spectral radius against $k_{d,tot}$ for the three axes.

What the cap costs

With the cap the barrier can no longer become infinite, so the controller on the X3 can no longer *force* the state to stay inside the tube no matter what; it can only push harder near the wall, up to the cap. Any guarantee of the paper is a continuous-time guarantee, and this is the price of running it on a real digital loop. The complete documentation measures how often the envelope is left in each experiment.

5.4 Block 10 — From thrust and torque to four rotor speeds

The idea in plain words

Each rotor pushes up with F_i. Pushing up at a point off the centre makes a torque: a rotor at lateral position y_i gives a torque $y_i F_i$ about x (roll); a rotor in front ($x_i > 0$) pitches it with $-x_i F_i$; and each rotor's drag twists the body about z with $-\mathrm{dir}_i c_Q F_i$. These four sums form a 4×4 matrix. Inverting it gives the four thrusts that produce exactly the requested thrust and torques.

Block 10: allocation (mixer) from the X3 geometry

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = A\boldsymbol{F}, \qquad A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ -0.22 & 0.20 & 0.22 & -0.20 \\ -0.13 & 0.13 & -0.13 & 0.13 \\ -0.016 & -0.016 & 0.016 & 0.016 \end{bmatrix}, \qquad \boldsymbol{F} = A^{-1} \begin{bmatrix} T \\ \boldsymbol{\tau} \end{bmatrix}, \quad \Omega_i = \sqrt{F_i / k_T} \tag{B10}$$

Derivation

$\boldsymbol{r}_i \times F_i \boldsymbol{e}_3 = [y_i F_i,\ -x_i F_i,\ 0]^\top$, and the yaw reaction of rotor i is $-\mathrm{dir}_i c_Q F_i$ (the plugin's sign convention). Summing over the rotors gives the rows of A with the rotor positions of the SDF: rotor 0 $(0.13, -0.22)$ ccw, rotor 1 $(-0.13, 0.20)$ ccw, rotor 2 $(0.13, 0.22)$ cw, rotor 3 $(-0.13, -0.20)$ cw. A is invertible, so any $(T, \boldsymbol{\tau})$ maps to unique thrusts; physically they must also satisfy $0 \le F_i \le k_T \cdot 800^2 = 5.471\,\mathrm{N}$. If not, our mixer first drops yaw, then scales roll and pitch down, so that thrust (staying in the air) has priority.

Toy example — hover and the torque the X3 can produce

Hover: $T = mg = 14.911\,\mathrm{N}$, $\boldsymbol{\tau} = 0 \Rightarrow F_i = 3.728\,\mathrm{N}$ each and $\Omega_i = \sqrt{3.728/8.54858 \times 10^{-6}} = 660.4\,\mathrm{rad/s}$.

Largest torque about one axis while hovering (each F_i must stay in $[0, 5.471]$): roll 1.464, pitch 0.907, yaw $\mathbf{0.106}\,\mathrm{N\,m}$. The paper's disturbance has a yaw amplitude of $0.15\,\mathrm{N\,m}$, more than the X3 can ever cancel: at full paper amplitude the yaw constraint is infeasible for *any* controller.

Toy example — what our first mixer did

The first node used a textbook "+/−" mixer, $m_i = \frac{T}{4k_T} \pm \frac{\tau_y}{4k_T l} \mp \frac{\tau_x}{4k_T l} \mp \frac{\tau_z}{4c_Q}$, whose signs do not match the X3's rotor numbering. Pushing it through A gives the torque produced per unit

commanded:

$$M_{v1} = \begin{bmatrix} 0.044 & -0.933 & 0 \\ 0 & 0 & 6.9 \times 10^{-5} \\ 0.071 & 0 & 0 \end{bmatrix} \qquad \text{(ideal: } I_3\text{)}.$$

A pitch command produced a *roll* torque, nothing produced a pitch torque, and the yaw term (divided by c_Q instead of $k_T c_Q$) was five orders of magnitude too small. With (B10), $M = A_{2:4} A_{:,2:4}^{-1} = I_3$ exactly.

MATLAB: `toy_model/step7_x3_mixer.m` builds A from the SDF, prints the hover speeds, the authority and M_{v1}, and plots M_{v1} next to the corrected mixer.

In our project

`allocation_matrix()` in `x3_sim/x3_model.py` builds A; the controller stores A^{-1} and the desaturation loop is at the end of `FixedController.update()`. The rotor speeds are published on `/X3/gazebo/command/motor_speed` by `gazebo.py`.

6 Where Every Equation Lives in Our Project

Equation	**Spacecraft reproduction** (`paper_reproduction.py`)	**X3** (`x3_sim/`, `gazebo.py`)	**MATLAB toy model** (`toy_model/`)
(1) plant	`rhs()`, RK4 1 ms	Gazebo; surrogate `X3.deriv()`	`step6` `rhs()`
(3)–(4) saturation	`sat_vec()`, 2.5 N m	clip 0.35 N m + mixer limits	`step3`, `step6`
(8) FTPPF	`ftppf()`	`controllers.ftppf()`	`lib/ftppf.m`, `step1`
(11) $\varepsilon_1, \varepsilon_2$	`control()`	`update()`	`lib/control_law.m`, `step2`
(12), (22) law	`control()`	`update()` + cap (B9)	`lib/control_law.m`, `step3`, `step5`
(14) adaptation	`simulate()`	`update()` (leakage floor)	`step4`
(15) Lyapunov	`lyapunov()`	–	–
(B7) body rate	–	`update()`, "F11"	–
(B8) outer loop	–	`update()`	–
(B9) gain cap	–	`update()`, "F15"	`step8`
(B10) mixer	–	`allocation_matrix()`, `FixedController`	`step7`

In the X3 column, `update()` is the method `FixedController.update()` of `x3_sim/controllers.py`, which `gazebo.py` calls on every pose message.

6.1 Which barrier form each file uses

File	Barrier	Role
`paper_reproduction.py`	paper, symmetric, none	reference results (all three selectable)
`x3_sim/controllers.py`, `gazebo.py`	paper, symmetric, none	X3 controller (ROS parameter `controller_mode`)
`toy_model/lib/control_law.m`	paper, symmetric, none	MATLAB toy model
`Drones_Mujoco_Pointing_Trial.ipynb`	paper	3-D MuJoCo attitude demos
`index.html`, `app.js`, `simulation.js`	symmetric	interactive browser demo (illustration only)
`legacy/gazebo_v1.py`	symmetric, ε clipped at 0.9	first Gazebo node (crashed; kept for the record)

6.2 From the equations to a running controller: one control step on the X3

Every pose message triggers, in this order:

1. read the stamp, position and quaternion; keep the quaternion sign continuous;
2. block 7: body rate from $\boldsymbol{q}_{k-1}^{*} \otimes \boldsymbol{q}_k$ and the stamp difference;
3. block 8: reference (hover or square), acceleration command, thrust T, desired attitude $\boldsymbol{q}_d$, errors $\boldsymbol{q}_e$, $\boldsymbol{\omega}_e$;
4. block 3: $\rho_q(t)$, $\rho_\omega(t)$; block 4: $\boldsymbol{\varepsilon}_1 = \boldsymbol{q}_{e,v}/\rho_q$, $\boldsymbol{\varepsilon}_2 = \boldsymbol{\omega}_e/\rho_\omega$;
5. block 9 (= blocks 5 and 6 with the cap): torque $\boldsymbol{u}$, update $\hat{\theta}$, clip to $\pm 0.35\,\mathrm{N\,m}$;
6. block 10: $\boldsymbol{F} = A^{-1}[T; \boldsymbol{\tau}]$ with thrust-priority de-saturation, $\Omega_i = \sqrt{F_i/k_T}$, publish.

This is the paper's Algorithm 1 (Remark 8), with steps 2, 3 and 6 added for the quadcopter.

A Recap, Glossary and References

A.1 Recap table

Block	Name	Key equation	One-line meaning
1	Rigid body	$\dot{\boldsymbol{q}}_v = \frac{1}{2}(q_4 I + \boldsymbol{q}_v^{\times})\boldsymbol{\omega}$, $J\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega}^{\times} J\boldsymbol{\omega} + \boldsymbol{u} + \boldsymbol{d}$	how torque turns the body
2	Saturation	$\mathrm{sat}(u_i) = u_i + \pi_i$, $\boldsymbol{T}_d = \boldsymbol{d} + \boldsymbol{\Pi}$	lost torque becomes one more unknown
3	FTPPF	$\rho = (\rho_0 - \rho_T(1 + t/T_s))e^{\kappa t/(t-T_s)} + \rho_T$	a tube that closes exactly at T_s
4	Normalised error	$\boldsymbol{\varepsilon}_1 = \boldsymbol{q}_v/\rho_q$, $\boldsymbol{\varepsilon}_2 = \boldsymbol{\omega}/\rho_\omega$	fraction of the tube in use
5	Control law	$\boldsymbol{u} = -k_p \boldsymbol{q}_v - k_d \boldsymbol{\omega} - [g\,\boldsymbol{s} + \hat{\theta}\,\mathrm{sat}(\boldsymbol{s}/\varsigma)]$	PD plus a damper that hardens at the wall
6	Adaptation	$\dot{\hat{\theta}} = (\lVert\boldsymbol{s}\rVert - \eta_2\hat{\theta})/\eta_1$	learn how hard the unknown pushes
7	Body rate	$\boldsymbol{\omega} = 2\angle(\boldsymbol{q}_{k-1}^{*} \otimes \boldsymbol{q}_k)/\Delta t$	rates from poses, in the body frame
8	Outer loop	$\boldsymbol{F}_d = m(\boldsymbol{a}_{cmd} + g\boldsymbol{e}_3)$, $\boldsymbol{q}_e = \boldsymbol{q}_d^{*} \otimes \boldsymbol{q}$	where to tilt to reach a position
9	Gain cap	$g_i \le g_{\max} J_i/J_z$	keep the 60 Hz loop stable
10	Mixer	$\boldsymbol{F} = A^{-1}[T; \boldsymbol{\tau}]$, $\Omega_i = \sqrt{F_i/k_T}$	four rotor speeds

A.2 Glossary

Attitude Orientation of the body with respect to the inertial frame.

Barrier function
A gain that grows without bound as a quantity approaches a limit, used to keep it away.

Boundary layer
Replacing $\operatorname{sgn}(s)$ by $\operatorname{sat}(s/\varsigma)$ to avoid chattering; costs exact convergence.

Chattering Fast switching of a discontinuous control (sgn) around zero.

FTPPF Finite-time prescribed performance function, Eq. (8).

Leakage The term $-\eta_2\hat{\theta}$ that keeps an adaptive estimate from growing without bound.

PPC Prescribed performance control: keeping errors inside predefined time-varying bounds.

Quaternion Four numbers $[\boldsymbol{q}_v; q_4]$, unit length, representing a rotation without singularities.

Saturation Actuator limit: the delivered torque is clipped at $\pm u_{\max}$.

SIL Software-in-the-loop: the real controller code driving a simulated vehicle.

Sliding variable
$\boldsymbol{s} = \boldsymbol{\omega} + c\boldsymbol{q}_v$; when it is zero the attitude error decays on its own.

Spectral radius
Largest $|\lambda|$ of a matrix; below 1 means a discrete-time loop is stable.

A.3 References

1. M. Golestani, K. A. Alattas, S. Ud Din, S. Mobayen, A. K. Alanazi, A. Fekih, "A Simple Structure Constrained Attitude Control for Rigid Bodies: A PD-Type Control," *IEEE Access*, vol. 10, pp. 10202–10209, 2022.
2. C. P. Bechlioulis, G. A. Rovithakis, "Robust adaptive control of feedback linearizable MIMO nonlinear systems with prescribed performance," *IEEE Trans. Autom. Control*, vol. 53, no. 9, pp. 2090–2099, 2008. (Origin of PPC.)
3. J. T.-Y. Wen, K. Kreutz-Delgado, "The attitude control problem," *IEEE Trans. Autom. Control*, vol. 36, no. 10, pp. 1148–1162, 1991. (Lyapunov function of Theorem 1.)
4. H. K. Khalil, *Nonlinear Systems*, 3rd ed., Prentice Hall, 2002. (Barbalat's lemma.)
5. Gazebo Sim, `quadcopter.sdf` world and `MulticopterMotorModel` system (X3 parameters).